

Write logic for this:

1. A retail chain wants to forecast product sales using factors like holidays, promotions, location, and inventory. The dataset has 80 features. How should they select the most impactful ones?

Answers:

This model involves forecasting product sales, to reduce number of features we can use Embedded Methods from Feature Selection method - Tree-Based Ensembles (like XGBoost or LightGBM) combined with Variance threshold for more accuracy.

Step 1: Clean and Prepare the Applicant Data

- Gather all raw data features from the credit applications
- Handle missing values and convert categorical data into numerical values.
- Split dataset into a Training Set to build the models and a test Set to test the accuracy.

Step 2: Apply Layer 1 – Variance threshold

- Drop columns where all rows have the exact same value. If a promotion was never run, its flag is useless.
- Set a threshold (e.g., 99%). Drop columns where 99% of the data contains the same value.
- Perform Correlation Analysis, by calculating a Pearson or Spearman correlation matrix. If two features correlate above 0.85, drop one to avoid redundancy.

Step 3: Apply Layer 2 – Tree-Based Ensembles (Non-Linear Interaction)

- Take the remaining, survival features from Step 2 and feed them into a tree-based model like XGBoost or LightGBM.
- Train these trees to split applicants into risk categories based on complex interactions
- Extract the Feature Importance Scores from the trained model. This ranks columns based on how much predictive accuracy or "gain" each feature added across thousands of decision trees

Step 4: Define the Final Threshold and Intersect

- Look at the ranked feature importance list from the tree-based model.
- Sort the 80 features from highest gain to lowest. Drop features that contribute to the bottom 5% of total predictive power.
- The remaining columns represent your optimized feature set.

Step 5: Train and Validate the Final Production Model

- Re-train the final model using only this highly refined, reduced set of columns.
 - Test the model on your unseen test Set to ensure it predicts the product sales accurately without overfitting.
 - Deploy the streamlined model. It will now run faster, require less data pipeline maintenance.
-

2.A podcast platform wants to recommend new podcasts to users based on the categories and hosts they follow. How should they design this content-based system?

Answers:

Content Based Recommendations:

- Gather the details for every podcasts and users in the database
 - Create Podcasts vector
 - Analyze the text features of the current user's historical podcast (hosts, categories)
 - Create users vector for all users
 - Using Dot matrix of user vector and podcasts vector, create Content vector
 - Rank the podcasts based on their scores.
 - Remove podcasts that are already present in the user's listening history.
 - Recommend the top-N podcasts with the highest similarity scores.
-

3.An industrial firm wants to predict machine failure using sensor readings and system logs (200+ features). How do they choose which inputs to use?

Answers:

This is a classification model.

- Collect the data
- Data cleaning like handling missing values, duplicate records and outliers
- Remove features that have the same value for almost every record (low variance).
- Use SelectKBest with an appropriate scoring function such as Mutual Information or ANOVA F-test to rank the features and retain the top K

- Evaluate feature importance from tree-based models like Random Forest or XGBoost, or use Recursive Feature Elimination if needed
 - Compare model performance before and after feature selection and choose the feature set that gives the best balance of accuracy, interpretability, and computational efficiency
-

4. A fashion app wants to suggest outfits to users based on their previous likes, color preferences, and seasonal trends. How should they implement the recommendation?

Answers:

Content Based Recommendations:

- Gather the details for every outfits and users in the database
 - Create Outfits vector
 - Analyze the text features of the current user's historical Outfit likes (color, preferences)
 - Create users vector of the outfits the user has liked
 - Using Dot matrix of user vector and Outfits vector, create Content vector
 - Rank the outfits based on their scores.
 - Remove outfits that are already present in the user's purchase history.
 - Recommend the top-N outfits with the highest similarity scores.
-

5. A research lab wants to predict heart disease using patient data. The dataset has lab results, habits, and vitals. How should they select only the most relevant features?

Answers:

This is a classification model.

- Collect the patients data
- Data cleaning like handling missing values, duplicate records and outliers
- Remove features that have the same value for almost every record (low variance).
- Use SelectKBest with an appropriate scoring function such as Mutual Information to rank the features and retain the top K
- Evaluate feature importance from tree-based models like Random Forest or use Recursive Feature Elimination if needed

- Compare model performance before and after feature selection and choose the feature set that gives the best balance of accuracy, interpretability, and computational efficiency.
-

6. A job portal wants to recommend job listings based on a user's resume and previous applications. How can they build a recommendation system?

Answers:

Content Based Recommendations:

- Gather the details for every job and user's resume
 - Create job vector
 - Analyze the text features of the current user's resume and previous job applications
 - Create a user profile vector by combining the resume features and the vectors of the jobs the user has previously applied to.
 - Calculate the similarity score using dot product between the user profile vector and every job vector.
 - Rank the jobs based on their scores.
 - Remove jobs that are already applied by the user.
 - Recommend the top-N jobs with the highest similarity scores.
-

7. A smart home system wants to predict energy usage using readings from multiple sensors. With 100+ features, how do they reduce complexity?

Answers:

This is a regression model.

- Collect the readings data
- Data cleaning like handling missing values, duplicate records and outliers
- Remove features that have the same value for almost every record (low variance).
- Use SelectKBest with an appropriate scoring function such as Mutual Information to rank the features and retain the top K
- Evaluate feature importance from tree-based models like Random Forest regressor or use Recursive Feature Elimination if needed

- Compare model performance before and after feature selection and choose the feature set that gives the best balance of accuracy, interpretability, and computational efficiency.

8. An e-learning platform wants to suggest tutorials based on a learner's skill gaps identified from quizzes. How should they build this system?

Answers:

Content Based Recommendations:

- Gather the details for every tutorials and user's skill gaps
- Create tutorials vector
- Analyze the text features of the current user's skill gaps
- Create a user profile vector by combining the user's skill gaps features and the vectors of the tutorials.
- Calculate the similarity score using dot product between the user profile vector and every tutorials vector.
- Rank the tutorials based on their scores.
- Remove tutorials that are already studied by the user.
- Recommend the top-N tutorials with the highest similarity scores.

9. A social media app wants to predict post engagement (likes/comments) using post features, timing, and user activity. How do they select the right features?

Answers:

This is a classification model.

- Collect the posts data
- Data cleaning like handling missing values, duplicate records and outliers
- Remove features that have the same value for almost every record (low variance).
- Use SelectKBest with an appropriate scoring function such as Mutual Information(`mutual_info_classif`) to rank the features and retain the top K
- Evaluate feature importance from tree-based models like Random Forest classifier or use Recursive Feature Elimination if needed
- Compare model performance before and after feature selection and choose the feature set that gives the best balance of accuracy, interpretability, and computational efficiency

10. Now you want to create an app named `attendance` in the project and register it. write logic for this Django project

Answer:

Streamlit Attendance Prediction Project steps:

- Create a project folder named Attendance_Prediction.
- Create a virtual environment and install the required libraries:
 - Streamlit
 - Pandas
 - NumPy
 - Scikit-learn
 - Joblib/Pickle
- Train your attendance prediction model using historical attendance data.
- Save the trained model (e.g., `attendance_model.pkl`) and any preprocessing objects such as a scaler or encoder.
- Create the main Streamlit application file (`app.py`).
- Load the saved model and preprocessing files inside `app.py`.
- Design the user interface using Streamlit widgets (text boxes, number inputs, select boxes, buttons) to collect inputs such as:
 - Student ID
 - Attendance percentage
 - Previous attendance
 - Number of leaves
 - Subject
 - Semester
 - Any other relevant features
- Preprocess the user inputs using the same transformations applied during model training.

- Pass the processed input to the trained model.
 - Display the predicted attendance result or risk level to the user.
 - (Optional) Add additional pages such as:
 - Dashboard
 - Reports
 - Attendance Analytics
 - Model Performance
-

HOPE AI